

Perl for Newbies

Michelle Shaw, ACEnet CRC
May 2010

Topics to cover

- intro...
- first program...
- functions, statements
- numbers, strings, quotes
- variables and arrays
- decisions
- loops
- file I/O
- some built-in functions
- more info

Intro...

- perl is used primarily as a scripting language
- powerful for several tasks:
 - system administration type tasks
 - wrapper for one or more programs
 - pre- and post-processing tasks that don't require significant computing resources
 - as a wrapper for command-line utilities
- downside: more exceptions than rules, so there's not much in the way of standards...
- lots of add-ons, code and free documentation
- is "nearly" portable between different OS

A First Perl Program...

- the famous "hello world" program

```
#!/usr/bin/env perl
```

```
use strict;
```

```
use warnings;
```

```
print "hello world from my first perl program!";
```

Perl Syntax and Format

- perl consists of a series of statements and functions
- the functions are the "commands" within perl
 - e.g. print, chomp, etc.
- the statements are the "meat" of the perl code
- statements don't need to start at any particular line
- the statement ends with a semicolon
- comments begin with a #
- no line continuation character - perl pays little heed to whitespace (i.e. tab, newline, space..)

Perl Data Types

- data types in perl:
 - strings
 - numbers
 - lists
- perl also uses quotes differently:
 - `'..'` - contents are literal
 - `"..."` - contents are interpreted
 - ``...`` - used for executing command-line utilities
- the things inside quotes that are interpreted include special characters and variables

Variables

- three types in perl:
 - scalars (individual items)
 - arrays (lists)
 - hashes (dictionaries)
- in this talk - work mainly with scalars
- scalars are assigned using:
 - `$var = value;`
- accessed using:
 - `$var` (e.g. `print "var is: $var \n";`)

Built-in variables

- perl has some handy built-in variables
- some commonly used ones are:
 - `$@` - error message from eval operator
 - `$_` - default input and pattern-searching space
 - `$a`, `$b` - special variables used with `sort()`
 - `@ARGV`:
 - array of inputs to the perl script
 - first argument is zero
 - `%ENV` - the current environment stored as a hash

Arrays/Lists

- arrays in perl are actually lists
- define them by prepending an "at" symbol (@) to the name of the array:
 - `my @array = values;`
- use the array values by prepending the name with a \$ and asking for the element in []:
 - `print "element 0 is $array[0] \n";`
- the number of array elements is accessed using one of two ways:
 - use the builtin scalar: `scalar @array`
 - grab last index and add 1: `$#array+1`

Special array - ARGV

- special built-in array: ARGV
- stores arguments to your perl script
- no need to assign or define it
- access it as you do any other array
- first element is at index zero
- an example:

```
my $narg = $#ARGV;  
if ($narg == 0) {  
    print "Usage: ./my_script.pl ...";  
}
```


Hashes

- hash is basically a list
- also called a "dictionary"
- similar to an array, but its elements are not numerically indexed
- instead, a "key" is used to store and later grab an element of a hash
- syntax:

```
%hashname = (  
    "key1" => "var1",  
    "key2" => "var2",  
    ....);
```

Hashes of Arrays

- hashes can also be used to store arrays of varying sizes
- we can key each array by its name, for example
- arrays don't need to be of the same size or type
- it is analogous to a struct in C or a type in fortran 90
- syntax:

```
%hashname = (  
    "key1" => [ "arr1val1", "arr1val2",...],  
    "key2" => [ "arr2val1", "arr2val2",...],  
    ....);
```


Hashes of Arrays - Example

```
#!/usr/bin/env perl
use strict;
use warnings;
my %jobdirs = (
    cm => [ "cpmd_jobs", "vasp_test" ],
    bioch => [ "gromacs_jobs", "namd_jobs" ],
    chem => [ "gauss_test", "nwchem_jobs", "gamess_jobs"]);
my $jobdir;
for my $jobtype (keys %jobdirs) {
    print "job type is $jobtype \n";
    for my $i ( 0 .. ${ $jobdirs{$jobtype} }) {
        $jobdir = '/home/dshaw/'.$jobdirs{$jobtype}[$i];
        print "$jobdir has: \n";
        system("ls -lrth $jobdir");
    }
}
```

Operators and Pattern Matching

- basic perl has basic mathematical and string comparison operators
- examples:
 - math: +, -, *, /, %
 - string comparison: ==, !=, >, <, =~
- perl also has an extensive regular expression library
- string matches are tested using the "=~" operator
- look at regular expressions a little later...

Decisions

- decisions in perl done using the if-else loop:

```
if (some condition is true)  
{  
    do something  
}  
else  
{  
    do something else  
}
```

Loops

- □ do a set of tasks, defined either by a decision or by a set number
- some commonly used loops in perl:
 - for/foreach
 - while
 - until

for/foreach

- □ syntax:

```
for (set of tasks) {  
    do some tasks  
}
```

- used as an iterator

- example:

```
for($i=1; $i<10; $i = $i + 1) {  
    print "I am at iteration $_. \n";  
}
```

for/foreach

- perl also uses autoincrement and autodecrement, so the following are equivalent:
 - $\$i = \$i + 1$ and $\$i++$
 - $\$j = \$j - 1$ and $\$j--$
- perl also has an equivalent to for called "foreach", useful for stepping through hashes
- syntax:

```
foreach(start..end) {  
    do something  
}
```


while

- used for doing some tasks based on a condition remaining valid

- syntax:

```
while (something is valid) {  
    do something  
}
```

- example:

```
while ($i < 10) {  
    print "now at $i, going to 10 \n";  
    $i++;  
}
```

until

- essentially a while loop but in reverse
- while - does something while a condition holds
- until - does something while a condition does not hold
- syntax:

```
until (condition is valid) {  
    do some work  
}
```


File I/O

- data sometimes read/written from/to files
- perl does file handling in a similar fashion to C
- process for file handling in perl:
 - open the file
 - read/write the file
 - close the file
- two perl built-in functions for file handling: open and close
- also have a builtin method for accessing file contents by line: <FILEHANDLE>

File I/O

- syntax of open:
 - open FH, "MODE", "filename";
 - FH is the "file handle" - used to access file
 - MODE is the file mode (similar to linux)
 - MODE is one of < (read), > (write), >> (append), +< (r/w and update), +> (rw/ but overwrite first)
 - filename is the name of the file (including path if in a different dir than the script)

File I/O

- syntax of close:
 - close FH;
- file lines are read using <FH>
- example of a read:
 - while (my @lines = <FH>) {
 - do something with the lines
 - }
- write to a file using print:
 - print FH "something to print";
 - same as what we've seen with print, just add the FH right after "print"

Built-in functions

- perl has several built-in utilities to use for a variety of tasks:
 - chomp - remove the newline character from the end of a line
 - split - split up a line into its parts, separated by a delimiter (usually a space or a tab)
 - die - exit the script with an error message
 - glob - expand filenames using wildcards
 - splice - add or remove elements in array
 - substr - get a portion of a string
 - use - use functionality from a module

Example using split, chomp, die, use

```
#!/usr/bin/env perl
use strict;
use warnings;
my $numArgs = $#ARGV + 1;
if($numArgs == 0) {
    die "ERROR: no username supplied! \nUSAGE: ./check_sgejob.pl
<username> \n";
}
...
foreach my $line (@qstatout) {
    chomp($line);
    if($line =~ /\b$username\b/) {
        my @jobspecs = split(" ", $line);
    }
    ...
}
```

More Info

- internet - perl.org, perldoc, etc.
- perl document pages online (man perldoc for usage)
- O'reilly books
- ACEnet support (support@ace-net.ca)
- your local CRC